

SOUFER Tools — Documentação do Front-end (Entrega P1)

Disciplina: Desenvolvimento de Interfaces de Usuário para Web **Projeto Integrador:** SOUFER Tools — controle web de retirada e devolução de ferramentas da manutenção da Soufer **Entrega:** P1 — Desenvolvimento da Interface do Projeto Integrado (02/10/2026)

Nome	RA
Guilherme Portilho da Rosa Santi	25000151
Henrique de Oliveira Molinari	25001176
João Augusto de Freitas	25000019
Kauan Leander Leandrini	25000795

1. Sobre a aplicação

O SOUFER Tools é o sistema de controle de retirada e devolução de ferramentas da manutenção da Soufer. Ele acompanha qual ferramenta está disponível, em uso ou indisponível, quem retirou o quê, e abre ocorrências automaticamente quando uma ferramenta volta avariada ou é dada como perdida.

Este documento cobre exclusivamente a parte de **front-end** (`/web` do monorepo), que é o escopo avaliado nesta entrega parcial (P1). A API RESTful e o banco de dados são desenvolvidos em paralelo por outra dupla da equipe e, conforme o próprio enunciado da atividade, **a integração real com a API não é exigida nesta etapa** — as telas usam dados simulados.

2. Aplicação publicada

Além de rodar localmente, o front-end está hospedado e acessível publicamente em:

<https://soufer-tools.servidorcaseiro.online/>

O deploy é feito a partir do build de produção (`npm run build`) do diretório `/web`, servido em um servidor próprio da equipe (fora da AWS, usado como ambiente de testes/demonstração enquanto a infraestrutura em nuvem definitiva não entra em produção — ver decisão arquitetural em `/CLAUDE.md` da raiz do repositório).

Como esta entrega não exige API real, a tela de login expõe um botão **"Entrar sem a API (demonstração)"**, que autentica um usuário fixo em memória (sem token, sem persistência) e leva direto ao dashboard — é assim que a aplicação publicada deve ser avaliada nesta etapa, sem precisar de um backend no ar.

3. Estrutura do projeto

O repositório é um monorepo com a API e o front-end em pastas separadas:

```
PI-2026.2/
├─ api/           # API Node.js/Express (fora do escopo desta entrega)
├─ docs/         # documentação técnica e de projeto
└─ web/          # front-end React (escopo desta entrega)
```

Dentro de `web/`, a estrutura de código segue organização por tipo/domínio:

```
web/
├─ src/
│  ├─ main.tsx      # entrypoint, importa index.css
│  ├─ App.tsx       # providers: Query, Router, Toaster
│  ├─ router.tsx    # todas as rotas (React Router 7)
│  ├─ index.css     # Tailwind v4 + tokens do design system
│  ├─ layouts/
│  │  └─ AppLayout.tsx # sidebar + cabeçalho (rotas autenticadas)
│  ├─ pages/        # uma página por rota
│  ├─ components/
│  │  ├─ ui/        # gerados pelo shadcn/ui
│  │  ├─ dashboard/ # KpiCard, BarraSetor, ...
│  │  ├─ fluxo/     # retirada/devolução
│  │  ├─ layout/    # CabecalhoApp, RodapeSidebar
│  │  └─ status/    # tela pública de consulta
│  ├─ hooks/        # hooks compartilhados
│  ├─ assets/sfx/   # efeitos sonoros (Vite)
│  └─ lib/
│     ├─ api.ts      # instância única do Axios
│     ├─ auth.tsx    # AuthProvider, sessão, rotas
│     ├─ som-confirmacao.ts # preferência em localStorage
│     └─ utils.ts    # helper cn() do shadcn
├─ public/brand/    # logos oficiais da Soufer
├─ components.json  # configuração do shadcn/ui
├─ .env.example     # variável VITE_API_URL
└─ vite.config.ts   # alias @ -> ./src
```

Cada camada acima tem um papel fixo: `pages/` nunca guarda lógica de negócio reutilizável, `components/` é sempre a extração de algo que já apareceu (ou tem chance de aparecer) em mais de uma tela, e `lib/` concentra integrações (HTTP, autenticação, som) que não são componentes visuais.

Cada tela nova nasce como página em `src/pages/`, mas qualquer bloco de interface repetido ou com chance de reuso é extraído para `src/components/` (ou uma subpasta de domínio) — foi assim que `StatusBadge`, `CampoPin` e `CampoComErro`, por exemplo, nasceram numa única tela e depois passaram a ser reaproveitados em outras.

4. Tecnologias utilizadas

Camada	Tecnologia
Build/dev server	Vite 8
Framework	React 19 + TypeScript
Roteamento	React Router 7
Dado remoto/cache	TanStack Query
Cliente HTTP	Axios
Formulários e validação	React Hook Form + Zod
Estilização	Tailwind CSS v4 (CSS-first, sem <code>tailwind.config.js</code>)
Componentes de UI	shadcn/ui (preset <code>radix-nova</code> , sobre Radix UI)
Gráficos	Recharts
Código de barras	react-barcode (Code128)
Tipografia	Geist Variable (<code>@fontsource-variable/geist</code>)
Lint	Oxlint

5. Como instalar e executar localmente

Pré-requisitos

- Node.js **≥ 22.12** (definido em `engines` no `package.json`)
- npm

Passo a passo

```
# 1. Clonar o repositório
git clone https://github.com/joaoaugusto-dev/PI-2026.2.git
cd PI-2026.2/web

# 2. Instalar as dependências
npm install

# 3. Configurar variáveis de ambiente
cp .env.example .env
# .env contém VITE_API_URL — pode manter o valor padrão mesmo sem
# a API no ar, já que a P1 é avaliada com dados simulados

# 4. Rodar em modo desenvolvimento
npm run dev
# abre em http://localhost:5173
```

Outros comandos disponíveis

```
npm run build      # gera o build de produção (tsc -b && vite build)
npm run preview    # serve o build de produção localmente
npm run lint       # roda o Oxlint
```

6. Como testar a aplicação

Sem precisar de API no ar (local ou em <https://soufer-tools.servidorcaseiro.online/>):

1. Abra a tela de **login** (`/login`) e use o botão "**Entrar sem a API (demonstração)**".
2. No **dashboard** (`/`), confira os KPIs, a lista de ferramentas por setor e os empréstimos atrasados.
3. Em **Retirada** (`/retiradas/nova`), percorra o fluxo de identificação de colaborador/ferramenta e confirmação.
4. Em **Devolução** (`/devolucoes`), busque um empréstimo simulado e teste os estados de devolução "OK", "Avaria" e "Perda" — os dois últimos abrem o formulário de ocorrência.
5. Em **Design System** (`/design-system`), veja a paleta, tipografia, estados de status/botão/campo e as animações do sistema documentadas na própria interface.
6. Em **Status** (`/status`), veja a tela pública de disponibilidade (base do quiosque de consulta).
7. As demais rotas do menu (Ferramentas, Colaboradores, Calendário, Cadastros auxiliares, Importar CSV, Histórico) ainda estão como placeholder — são telas previstas no roteamento, mas com implementação prevista para sprints seguintes.

Toda tela concluída foi validada em três larguras — **360px, 768px e 1280px** — antes de ser considerada pronta, conforme a regra de responsividade do guia do projeto.

7. Decisões tomadas no desenvolvimento do front-end

Estas são as decisões de maior impacto tomadas pela equipe ao longo do desenvolvimento do `/web`, registradas para justificar escolhas que fogem do "caminho óbvio":

- **Design system definido em código, não no Figma.** Paleta, tipografia, espaçamento e movimento foram fechados direto em `src/index.css` (tokens CSS dentro de `@theme inline` do Tailwind v4), e documentados numa página própria da aplicação (`/design-system`) em vez de uma página separada no Figma — assim o design system nunca fica dessincronizado do que está realmente implementado.
- **shadcn/ui com preset radix-nova.** A CLI v4 do shadcn oferece um preset "Base UI" por padrão; a equipe manteve o preset sobre Radix UI para ficar de acordo com a stack já decidida no planejamento do projeto.
- **Tailwind v4 CSS-first**, sem arquivo `tailwind.config.js` — todos os tokens de tema vivem em `src/index.css`.
- **Login e cadastro por matrícula + senha numérica, nunca e-mail.** O ambiente é fabril e o operador não tem e-mail corporativo no dia a dia. A senha é digitada num componente de PIN dedicado (`CampoPin`, 6 dígitos, estilo lock screen de celular) em vez de um input de senha tradicional.
- **Sessão do usuário logado persiste em localStorage por 7 dias.** A especificação original previa sessão só em memória, mas isso forçava login toda vez que a aba fechava — inviável no uso diário da manutenção. Já a sessão de **consulta pública (quiosque)** permanece sem qualquer persistência, com 15 minutos de duração — por design, para não deixar sessão aberta num terminal compartilhado.
- **Tema claro na tela de login, tema escuro (negativo) na sidebar.** A sidebar é ambiente administrativo fixo e usa sempre o visual escuro com logo branco; o login, por ser a primeira tela vista em chão de fábrica sob luz industrial forte, usa tema claro — decisão tomada depois de um relato real de dificuldade de leitura em campo.
- **StatusBadge como único componente de status do sistema.** Cada estado (disponível, em uso, indisponível, atraso) tem cor e forma própria (círculo cheio, círculo vazado, quadrado, triângulo), para que o status nunca dependa só da cor — resolve também a legibilidade sob luz industrial e para usuários com daltonismo.
- **Calendário construído do zero** (`SeletorDataCalendario`) em vez de usar uma biblioteca de date picker. O motivo foi de comportamento de layout: um popover flutuante de terceiros ficava atrás de rodapés `sticky` do restante da interface, um problema de empilhamento (stacking context) que nenhum `z-index` resolvia de fora; a solução foi abrir o calendário em fluxo normal, empurrando o conteúdo abaixo, sem posicionamento absoluto.

- **Regras de movimento/animação centralizadas em tokens no `index.css`**. Toda animação do sistema só anima `transform` e `opacity` (as únicas propriedades que o navegador resolve sem recalculando layout), garantido por um script (`npm run check:motion`) que falha o build se algum keyframe animar propriedades de layout como `width` / `top` / `box-shadow`. Também há suporte automático a `prefers-reduced-motion`.
- **Modo demonstração no login, específico para esta entrega (P1)**. Como a avaliação da interface não exige integração real com a API, foi adicionado um atalho de login (`entrarComoDemo()`) que não depende de backend — sem ele, qualquer pessoa que abrisse o repositório ou o link publicado veria só a tela de login sem conseguir navegar pelo resto da aplicação. Será removido quando a integração real com a API estiver concluída.
- **Som de confirmação opcional**, ligado por padrão e persistido em `localStorage`, tocado a cada ação concluída com sucesso (retirada, devolução, cópia de dado) — pensado para um operador que não está olhando constantemente para a tela.
- **Toda chamada HTTP centralizada em `src/lib/api.ts`** (uma única instância do Axios), nunca `fetch` solto ou instância nova por arquivo — facilita trocar a URL da API e tratar erros de autenticação (interceptor de 401) num único lugar.

8. Estado atual e próximos passos

Nesta entrega, a interface está estruturada em componentes reutilizáveis, navegável via React Router, com formulários validados (React Hook Form + Zod), responsiva nas três larguras exigidas, e com interações funcionais (ações de botão, exibição condicional, dados simulados) — exatamente o escopo previsto para a P1.

Os próximos passos, fora do escopo desta entrega, são: integração real com a API RESTful (`/api`) substituindo os dados simulados, conclusão das telas ainda como placeholder, e o deploy final em nuvem (AWS S3 + CloudFront), com o endereço atual em `servidorcaseiro.online` servindo como ambiente de demonstração até lá.